HTML & CSS Interview Questions

Simple definitions, examples, interview-ready explanations, and important corrections to the handwritten notes

PART 1 — HTML

1. Semantic HTML elements

Semantic elements are HTML elements whose names describe the meaning or purpose of their content. They help browsers, developers, search engines, and assistive technologies understand the page structure.

```
Example: <header>, <nav>, <main>, <article>, <section>, <aside>, <footer>
```

Interview tip: Use semantic elements when their meaning fits the content. Do not choose them only for their default appearance.

2. HTML5

HTML5 is the modern major version of HTML. It introduced useful semantic elements, native audio and video support, improved forms, and APIs that make web pages more capable.

```
Example: <video controls><source src='movie.mp4' type='video/mp4'></video>
```

3. DOCTYPE

The DOCTYPE declaration tells the browser which document standard to use. In modern HTML, the short declaration below tells the browser to use standards mode. It is a declaration, not an HTML element or tag.

```
Example: <!DOCTYPE html>
```

Interview tip: It does not literally mean 'the browser is using the latest HTML automatically'; it declares an HTML document and helps prevent quirks mode.

4. HTML element

An element is the complete HTML unit: usually an opening tag, content, and a closing tag. Some elements are void elements and do not have content or a closing tag.

```
Example: <p>Hello</p> — the complete paragraph element.
```

5. HTML tag

A tag is the markup written inside angle brackets that marks the beginning or end of an element. Tags are the pieces used to create elements.

```
Example: <p> is the opening tag; </p> is the closing tag.
```

6. HTML attributes

Attributes provide extra information or configuration for an element. They are normally written in the opening tag as name-value pairs.

```
Example: <a href='https://example.com' target='_blank'>Visit</a> — href and target are
attributes.
```

7. Types of attributes

Two useful groups are: (1) global attributes, which can be used on most HTML elements, such as id, class, title, lang, and data-*; and (2) element-specific attributes, which apply to particular elements, such as href on <a>, src on <img>, and type on <input>.

```
Example: <div id='intro' class='card' title='Introduction'>...</div>
```

8. Types of HTML tags / elements

A beginner-friendly grouping is: (1) paired/container elements, which have opening and closing tags and can contain content; and (2) void elements, which do not have closing tags and cannot wrap content. 'Self-closing' is often said informally, but HTML void elements are not XML-style self-closing elements.

```
Example: Paired: <h1>Title</h1>, <div>Text</div>
Void: <br>, <hr>, <img>, <input>
```

9. input type='submit' vs button type='button'

An <input type='submit'> creates a submit control that submits its form. A <button type='button'> is a normal button and does not submit the form by itself; it is commonly used with JavaScript. A <button type='submit'> also submits a form.

```
Example: <form>
  <input type='submit' value='Submit form'>
  <button type='button'>Open menu</button>
  <button type='submit'>Save</button>
</form>
```

Interview tip: Inside a form, a <button> with no type generally behaves as a submit button. Set the type explicitly to avoid surprises.

PART 2 — CSS

10. Block-level vs inline elements

Block-level elements normally start on a new line and take the available width by default. Inline elements flow within a line of text and take only the space their content needs. CSS display values can change this behaviour.

```
Example: Block examples: <div>, <p>, <section>. Inline examples: <span>, <a>, <strong>.
```

Interview tip: Width and height normally work as expected on block elements. On ordinary non-replaced inline elements, width and height do not set the box dimensions.

11. Inline, internal, and external CSS

Inline CSS is written in an element's style attribute. Internal CSS is written inside a <style> element, usually in the document head. External CSS is stored in a separate .css file and linked with <link>. External CSS is usually best for reusable, maintainable styling.

```
Example: Inline: <p style='color: blue;'>Hello</p>
Internal: <style> p { color: blue; } </style>
External: <link rel='stylesheet' href='style.css'>
```

12. CSS selectors: id vs class

A selector chooses which HTML elements receive a CSS rule. An id selector uses # and is intended to identify one unique element in a page. A class selector uses . and can be reused on many elements. An element can have multiple classes.

```
Example: #header { color: navy; }
.card { padding: 12px; }
<div id='header' class='card'>Welcome</div>
```

Interview tip: An id should be unique within the page; a class is reusable. Specificity also differs: an id selector has higher specificity than a class selector.

13. CSS box model

The box model describes how an element's size is calculated. From inside to outside it contains content, padding, border, and margin. Padding is inside the border; margin is outside it.

```
Example: div { width: 200px; padding: 10px; border: 2px solid; margin: 15px; }
```

Interview tip: With the default content-box model, the declared width applies to the content only. With box-sizing: border-box, the declared width includes content, padding, and border.

14. Flexbox and Grid

Flexbox is a one-dimensional layout system: it arranges items along one main axis, either a row or a column. Grid is a two-dimensional layout system: it controls rows and columns together. Use Flexbox for a row of buttons or navigation; use Grid for a page or card layout.

```
Example: Flexbox: .container { display: flex; gap: 12px; align-items: center; }
Grid: .container { display: grid; grid-template-columns: repeat(3, 1fr); gap: 12px; }
```

15. CSS position

The position property controls how an element is placed. static is the default. relative keeps the element in normal flow but lets you offset it. absolute positions it relative to its nearest positioned ancestor. fixed is positioned relative to the viewport. sticky behaves like relative until a scroll threshold is reached, then sticks within its scrolling context.

```
Example: .box { position: relative; top: 10px; left: 5px; }
```

16. Media queries and breakpoints

A media query applies CSS only when a condition is true, commonly when the viewport width is within a range. Breakpoints are chosen widths where the layout changes to fit the available screen. This is a key technique for responsive design.

```
Example: @media (max-width: 600px) {
  .container { grid-template-columns: 1fr; }
}
```

Interview tip: There are no compulsory universal breakpoints. Choose breakpoints based on where your own layout needs to change.

17. Universal selector

The universal selector * matches every element. It is often used for a small reset or to apply box-sizing consistently.

```
Example: * { box-sizing: border-box; }
```

PART 3 — PRACTICAL TASK: BUILD A SIMPLE TEMPLATE

A common beginner task is to create a page with three cards or boxes in one row on a larger screen and stack them on a small screen. This example uses semantic HTML, a reusable class, Flexbox, and a media query.

```
<!DOCTYPE html> <html lang="en"> <head> <meta charset="UTF-8"> <meta name="viewport"
content="width=device-width, initial-scale=1.0"> <title>Three Cards</title> <style> * {
box-sizing: border-box; } .container { display: flex; gap: 16px; justify-content: center; }
.card { flex: 1; padding: 24px; border: 1px solid #999; text-align: center; } @media
(max-width: 600px) { .container { flex-direction: column; } } </style> </head> <body> <main
class="container"> <div class="card">Box 1</div> <div class="card">Box 2</div> <div
class="card">Box 3</div> </main> </body> </html>
```

QUICK REVISION — ONE-LINE ANSWERS

Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': '**Question**' 'frags': [ParaFrag(__tag__='b', bold=1, fontName='Helvetica-Bold', fontSize=9, greek=0, italic=0, link=[], rise=0, text='Question', textColor=Color(0,0,0,1), us_lines=[])] 'style': 'bulletText': None 'debug': 0) #Paragraph	Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': '**Remember this**' 'frags': [ParaFrag(__tag__='b', bold=1, fontName='Helvetica-Bold', fontSize=9, greek=0, italic=0, link=[], rise=0, text='Remember this', textColor=Color(0,0,0,1), us_lines=[])] 'style': 'bulletText': None 'debug': 0) #Paragraph
Semantic HTML	Elements describe the meaning of the content.
DOCTYPE	Declares the document type; modern HTML uses <!DOCTYPE html>.
Element vs tag	An element is the complete unit; a tag is the markup that starts or ends it.
Attribute	Extra information written in an element's opening tag.
Global attribute	An attribute usable on most elements, e.g. id or class.
Void element	An element with no closing tag or child content, e.g. img or br.
Submit vs button	Submit sends the form; type=button does not submit it by default.
Block vs inline	Block normally starts a new line; inline flows within text.
id vs class	id identifies one element; class can be reused.
Box model	Content + padding + border + margin.
Flexbox vs Grid	Flexbox is one-dimensional; Grid is two-dimensional.
Media query	Applies CSS when a screen or device condition matches.
Universal selector	* selects all elements.

Study suggestion: First learn the definition, then say one example aloud, and finally write the small code example without looking. That is a simple way to prepare for an HTML/CSS fresher interview.